

Virtualization

Processes

Program: A passive collection of instructions

Process: The abstraction provided by the OS of a running program

Machine State: What a program can read and change when it is running (registers, address spaces, open files, etc.)

Process Control Block (PCB): A data structure that contains the state of a process.

Creation of A Process by OS:

- Load data from disk to memory
- Allocate space for the run-time stack and initialize the stack with arguments (i.e. fill in the parameters for argc and argv)
- Allocate memory for program’s heap. Initially small, but OS may grow the heap as needed.
- Setup initial file descriptors (stdin, stdout, stderr).
- Transfer control of the CPU to the newly-created process (i.e. main()).

Context Switch: the CPU stops running one process (or thread) and starts running another

- Process A executes
- Hardware: generates timer interrupt, save regs(A) to kernel stack, move to kernel mode and jump to trap handler
- OS: Handle the trap, call switch routine, save regs(A) → proc_t(A), restore regs(B) ← proc_t(B), switch to k-stack(B), return-from-trap (into B)
- Hardware: restore regs(B) from kernel stack, move to user mode and jump to process B
- Process B executes.

CPU Scheduling

Running Time Metrics:

- $T_{\text{turnaround}} = T_{\text{completion}} - T_{\text{arrival}}$
- $T_{\text{response}} = T_{\text{first run}} - T_{\text{arrival}}$

FIFO/FCFS: First Come First Served, nonpreemptive

SJF: Shortest Job First, nonpreemptive

STCF: Shortest Time to Completion First, preemptive. Always run job that will complete the quickest

MLFQ: Multi-Level Feedback Queue, preemptive

1. If Priority (A) > Priority (B) then A runs
2. If Priority (A) == Priority (B) then A&B run in RR
3. Processes start at top priority
4. Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced

5. After some time period S, move all the jobs in the system to the topmost queue.

Lottery Scheduler: Randomly selects the next process to run based on ticket probabilities, giving each process CPU time proportional to its number of tickets.

- Ticket Currency: allows a user to allocate tickets among their running processes.
- Ticket Transfer: allows a process to temporarily hand off its tickets to another Process.
- Ticket Inflation: trusted processes can boost tickets to indicate its need for more CPU time.

Unfairness Metric: $U = \frac{T(\text{process1 completion})}{T(\text{process2 completion})}$

Stride Scheduler

- Each process is assigned a stride, which is the inverse proportion to the number of tickets the process has.
- Every time a process runs, its pass value is incremented by its stride.
- Scheduler selects the process with the smallest pass value.

The Linux Completely Fair Scheduler (CFS):

- Divide a time length evenly among n processes.
- Each process has a vruntime and a nice value.
- The process with the smallest vruntime is selected to run.
- Update vruntime of the running process by

$$vruntime_i += \frac{weight_0}{weight_{nice_i}} \times runtime_i$$

- Ready jobs’ vruntime are kept in a red-black tree.
- When jobs wake up, their vruntime is set to the minimum value in the tree.

Virtualizing Memory

Transparency: Process is unaware of sharing

Static Allocation: Randomly writes each program as it is loaded and placed in memory

- No Protection
- Cannot move addresses space after it has been placed.

Dynamic Allocation: Allocates memory at run-time

- Requires hardware support (Memory Management Unit)
- MMU dynamically changes process address at every memory reference

Sparse Allocation: Allocates memory in chunks, only allocate physical memory when needed.

Base+Bounds:

- MMU compares logical address to bounds register.
- If logical address is out of bounds, raise an error.

- Otherwise, add base register to logical address to get physical address.
- OS sets registers when loading process.
- Process can be moved by updating its base register.

Running Process with Base+Bounds:

- OS: allocate memory in process table, alloc memory for process, set base and bounds registers, then return from trap.
- Hardware: Restore registers, move to user mode, jump to process’s PC (stores the next instruction address).
- Process A: fetch instruction
- Hardware: translate VA, perform fetch.
- Process A: execute instruction.
- Hardware: if explicit load/store, ensure address is legal and translate the VA.

Segmentation: Divide the address space into segments (Code, Stack, Heap), each segment has separate base+bounds registers and grows independently.

- Explicit Approach: top bits of address select the segment, remaining are the offset.
- Implicit Approach: entire logical address is the offset, the corresponding segment is determined by how logical address is formed:
 - Formed from PC (Program Counter): code segment.
 - Formed from SP (Stack Pointer): stack segment.
 - Anything else: heap segment.

Segment	Base	Size	GrowsPositive?	Protection
00	32 K	2 K	1	R-X
01	34 K	3 K	1	R-W
11	28 K	2 K	0	R-W

Table 1: Segment Register

Paging Divide virtual and physical memory into fixed-size pages Map virtual pages to physical pages with a page table

TLB Contents

- **VPN:** used for lookup
- **PFN:** change the Virtual address VPN to PFN
- **G:** global bit (shared by all processes, don’t check ASID)
- **ASID:** Address Space Identifier (which process’s Page Table)
- **D:** dirty bit (changed when page has been written to)
- **V:** valid bit (valid translation present in entry)

Sapping Policy:

- **OPT:** Evict the page that will not be used for the longest time.
- **LRU:** Evict the page that has not been used for the longest time.

- **Random:** Randomly select a page to replace.

Clock Algorithm: Approximating LRU

- Add use bit to PTE, whenever page is referenced, bit set to 1
- Imagine all the pages of the system arranged in a circular list
- A clock hand points to some particular page, P
- When replacement needs to happen, OS checks use bit of page P
- if 1 , (not good candidate) set use bit to 0 and advance P , keep looking
- if 0 , (good candidate) replace this page

2-level Multi-level Paging Example:

```

VA = 0x0214 (15 bits -> 5|5|5)
+-----+-----+-----+
|DirIdx=0x00 |PTIdx=0x10 |Off=0x14 |
+-----+-----+-----+
|
PDBR=13 --> Physical Page #13 (Page Directory)
| Read byte 0 = PDE=0x83
v (V=1, Page Table PFN=0x03)
Physical Page #3 (Page Table)
| Read byte 16 = PTE=0x8E
v (V=1, Data PFN=0x0E)
Physical Page #14 (Data Page)
| Offset 0x14
v
Physical Address = (0x0E<<5) | 0x14 = 0x1D4

```

VAX/VMS Virtual Memory Layout:

- Page 0 invalid
- Segmentation: P0, P1, S
 - P0, P1: User segments
 - S: System segments(Kernel)
- Context Switch changes P0 & P1 PT Registers

Segmented FIFO

- RSS (Referenced Set Size): the maximum number of pages in memory for each process
- FIFO: first-in pages are moved to two **global** second-chance lists before actual eviction:
 - Clean-Page Free List
 - Dirty-Page List
- If another process needs free page, take first page off clean list
- If original process needs page before actual eviction, re-claims it from list.
- As the global list grow, it performs similar to LRU.
- Uses Clustering of pages from dirty list to write to disk.

Lazy Optmization: Demand Zeroing: To prevent process read sensitive data from previous process, the OS first mark PTE invalid.

Only on page fault (the process trying to use the page), the OS will zero the page.

Copy-On-Write: Share physical page across different processes. If one is writing, then copy the page and write to the new page.

ASLR: Address Space Layout Randomization: Randomize the address space layout of the process to avoid buffer overflow attacks.

Concurrency

Threads and Locks

Threads: Multiple threads of a process share:

- Process ID (PID)
- Address Space: Code and Heap
- Open file descriptors
- Current working directory
- User and group IDs

Each thread has its own:

- Stack for local variables and return addresses
- Set of registers, including program counter and stack pointer
- Thread ID (TID)

Atomic Hardware Operation:

For x86 systems:

- `xchg(dst, src)`: exchanges the value of `dst` with `src`. Returns the original value of `dst`.
- `CompareAndSwap(dst, expected, new)`: if the value of `dst` is equal to `expected`, exchanges the value of `dst` with `new`. Returns the original value of `dst`.

For ARM systems:

- `LoadLinked(*ptr)`: loads `ptr` and monitors for changes.
- `StoreConditional(*ptr, value)`: if the value of `ptr` has not changed since the last `LoadLinked`, then stores the value of `value` to `ptr` and returns 1. Otherwise, returns 0.

Lock Implementation:

With Atomic Exchange (Xchg):

```
typedef struct __lock_t {
    int flag;
} lock_t;
void init(lock_t *lock) {
    lock->flag = 0;
}
void lock(lock_t *lock) {
    while (Xchg(&lock->flag, 1) == 1) {
        // spin-wait (do nothing)
    }
}
void unlock(lock_t *lock) {
    lock->flag = 0;
}
```

With Compare and Swap (CAS):

```
void lock(lock_t *lock) {
    while (CompareAndSwap(&lock->flag, 0, 1) != 0) {
        //spin
    }
}
```

Condition Variables and Semaphores

Condition Variables: `wait(&c, &m)`: Wait for condition `c` to be signaled. It assumes the mutex `m` is locked when called. The call releases the lock and puts the thread to sleep.

After another thread calls `signal(&c)`, the thread will wake up and re-acquire the lock.

Meaning of Signal:

- Mesa Semantics: When a thread signals a condition, the waiting thread is only marked as ready to run.
- Hoare Semantics: When a thread signals a condition variable, it immediately transfers control to a waiting thread.

```
//Mesa Semantics
pthread_mutex_lock(&mutex);
while (!condition)
//The condition might still be false
//when the thread wakes up
pthread_cond_wait(&cond, &mutex);
```

Semaphores: A semaphore is an object with an integer value that can be manipulated with two atomic routines:

```
int sem_wait(sem_t *s) {
    // decrement the value of semaphore s by one
    // wait if value of semaphore s is negative
}
int sem_post(sem_t *s) {
    // increment the value of semaphore s by one
    // if there are threads waiting, wake one
}
```

Zemaphore implementation:

```
typedef struct __Zem_t {
    int value;
    pthread_cond_t cond;
    pthread_mutex_t lock;
} Zem_t;

void Zem_init(Zem_t *s, int value) {
    s->value = value;
    Cond_init(&s->cond);
    Mutex_init(&s->lock);
}

void Zem_wait(Zem_t *s) {
    Mutex_lock(&s->lock);
    while (s->value <= 0)
        Cond_wait(&s->cond, &s->lock);
    s->value--;
    Mutex_unlock(&s->lock);
}

void Zem_post(Zem_t *s) {
    Mutex_lock(&s->lock);
    s->value++;
    Cond_signal(&s->cond);
    Mutex_unlock(&s->lock);
}
```

Binary Semaphore: A semaphore that can only be 0 or 1.

- `sem_wait`: If value = 1, set it to 0 and proceed. If value = 0,

block until it becomes 1.

- `sem_post`: If threads are waiting, wake one of them (value stays 0). Otherwise, set value = 1.

Producer/Consumer Using Semaphores:

```
sem_t empty, full, mutex;
void *producer(void *arg) {
    int i;
    for(i=0;i<loops;i++) {
        sem_wait(&empty);
        sem_wait(&mutex);
        put(i);
        sem_post(&mutex);
        sem_post(&full);
    }
}

void *consumer(void *arg) {
    int i=0;
    for(i=0;i<loops;i++) {
        sem_wait(&full);
        sem_wait(&mutex);
        int tmp = get();
        sem_post(&mutex);
        sem_post(&empty);
        printf("%d\n",tmp);
    }
}

int main(int argc, char *argv[]) {
    // ...
    sem_init(&empty, 0, MAX);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    // ...
}
```

Concurrency Problems

Conditions for Deadlock:

- Mutual exclusion: Threads claim exclusive control of resources that they require.
- Hold and wait: Threads hold resources while waiting for other resources.
- No preemption: Resources cannot be forcibly removed from threads
- Circular wait: Circular chain of threads hold resources that other threads are waiting for.

Prevention Technique: Violating No Preemption

```
top:
    pthread_mutex_lock(L1);
    if (pthread_mutex_trylock(L2) != 0) {
        pthread_mutex_unlock(L1);
        goto top;
    }
```

Problem: LiveLock. Two threads may repeatedly attempt to acquire locks held by each other and repeatedly fail.

Persistence

I/O and Disk Devices

Os-Device Communication:

- **Polling:** The OS repeatedly checks the status of an I/O device.
- **Programmed I/O:** The main CPU controls data movement.
- **Hardware Interrupts:** The device sends an interrupt to the CPU when it is ready.

Methods of I/O interactions:

- **Explicit I/O Instructions:** OS controls register with data, and knows specific port which names the device, issues instruction.
- **Memory-Mapped I/O:** Device appears as memory location. OS can read/write to it like memory. A **device driver** is needed for each device, the OS just interacts with the driver.

Hard Disk Mechanics:

- **Seek:** The head moves to the correct cylinder.
- **Rotation:** The platter rotates under the head.
- **Transfer:** Data is transferred to/from the buffer.

Estimating I/O time:

T_I / O = T_seek + T_rotation + T_transfer

- Average seek time: 1/3 of the maximum seek time.
- Average rotation time: 1/2 of the rotation period.

I/O Schedulers:

- **SSTF:** always choose the request with the least seek time.
- **SCAN:** the disk arm sweep back and forth through cylinders.
- **SPTF:** cylinders rotate in one direction, choose the request for the sector that will pass under the disk head first.
- **Non-Work-Conserving:** wait for a few milliseconds to let other requests come in.

File Systems

File APIs:

- Path: a string that identifies a file.
- File descriptor: an integer that identifies an open file. Each process has its own file descriptor table managed by the OS.
- Inode number: a unique identifier for a file.

Inode: contains the metadata for a file or directory:

- type - regular file, directory, etc.
- size - the number of bytes in the file
- blocks - number of blocks allocated to file
- protection information - Who owns the file and who can access it

- time information - last accessed time, creation time, last modified time
- location information - Where data blocks reside on disk

Hard Links: create another name to the same inode number. If an inode has no more references, it is deleted.

Symbolic Links: a file that contains linking information to another file. If the underlying file is deleted, the symbolic link becomes a dangling reference.

Superblock: contains information about the file system:

- Number of inodes and data blocks.
- Where the inode table begins.
- Magic number indicating the file system type.

The Multi-Level Index:

- Direct pointer: refers to one disk block that belongs to the file. Inodes often contain 12 direct pointers.
- Indirect pointer: refers to a disk block that contains pointers to other disk blocks.

The maximum file size with 12 direct pointers and 1 indirect pointer is (12 + 1024) * 4KB = 4414KB. Doubly or Triply indirect pointers are used for larger files.

Opening, reading, creating and writing a file:

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]
open(bar)			read	read		read	read	
read()				read	write		read	

	data bitmap	inode bitmap	root inode	foo inode	bar inode	root data	foo data	bar data [0]
create (/foo/bar)		read write	read	read		read	read write	
write()	read write			read	write		write	

Fast File Systems

Fast File Systems: uses per-group super-block, inode bitmap, data bitmap, and data blocks to reduce seek time while large files are spread across multiple blocks.

fsck: Upon system reboot, run `fsck` to check for consistency.

Journaling: Before updating the disk, first write down a journal:

1. Journal write - Write out TxB, I[v2], B[v2], and Db , wait for these to complete
2. Journal commit - Write out TxE, wait for this to complete, transaction is committed
3. Checkpoint - Write the contents of the update to their final on-disk locations

Metadata Journaling:

1. Data write: Write data to final location; wait for completion.
2. Journal metadata write: Write the begin block and metadata to log; wait for completion
3. Journal commit: Write the transaction commit block (containing TxE)
4. Checkpoint metadata: Write the contents of the metadata to final locations
5. Free: Later, mark the transaction free in journal superblock

Log File Systems Use disk purely sequentially.

Reading in LFS:

1. Read the Checkpoint region.
2. Read all imap parts and caches in memory.
3. To read a file:
 - (a) Look up inode in imap.
 - (b) Read the inode.
 - (c) Read the data block.

Garbage Collection: Segment Summary is stored at the beginning of a segment, contains reverse pointers to the inodes for every data block in that segment. If the inode no longer points to the data block, then the block is garbage.

Crash Recovery:

- The pointers to imap pieces are occasionally saved to checkpoint regions.
- LFS has two checkpoints, only overwrite one at a time. The OS can identify the newest checkpoint using checksum or timestamps.
- Upon recovery, the OS read the newest checkpoint to find most of the imap pointers and segment tail, then find rest of the imap pointers by reading past tail.

SSDs

SSD Operations: (Each block has multiple pages.)

- Read a page
- Erase a block
- Program (write) a page

Flash Translation Layer (FTL):

- Translate reads/writes to logical blocks into reads/erases/programs.
- Reduce write amplification.
- Implement wear leveling to extend the lifespan of the SSD.

Virtual Machines Monitors

Virtual Machine:

- Language-based virtual machine, e.g. Java Virtual Machine.
- Lightweight virtual machines: rely on combination of hardware and software mechanisms to isolate applications, e.g. docker.
- System-level virtual machines: the isolated environment resembles the hardware of a OS.
 - Machine Simulator: A user-level application providing accurate simulation.
 - Hypervisor: Relies upon limited direct execution for maximum efficiency, relying upon traps which are emulated by the hypervisor.
 - * Bare-metal Hypervisor (type-1): in direct control of all resources of the physical machine.
 - * Hosted Hypervisor (type-2): operates as part of or on top of an existing host OS

Processor Status Word: consists of the tuple (M, B, L, PC) :

- the execution level $M = \{s, u\}$ (superuser or user mode)
- the segment register (B, L) ; (Segmented Memory Model)
- the current program counter (PC) , a virtual address

Popek / Goldberg Theorem: A VMM, relying exclusively on direct execution, may be constructed if the set of sensitive instructions for a computer is a subset of the set of privileged instructions.

$$\{\text{control-sensitive}\} \cup \{\text{behavior-sensitive}\} \subseteq \{\text{privileged}\}$$

- control-sensitive: instructions that can update the system state.
- behavior-sensitive: instructions whose behavior depends upon the system state.
- privileged: instructions that can only be executed in supervisor (i.e. kernel) mode and causes a trap when attempted from user mode.

Criteria for Virtualization:

- Equivalence - The exposed resource is equivalent with the underlying computer.
- Safety - Isolation requires that the virtual machines are isolated from each other as well as from the hypervisor.
- Performance - The virtual system must show at worst a minor decrease in speed.

Multiprocessor Scheduling:

SQMS: reuse single processor scheduling by putting all jobs that need to be scheduled into a single queue.

- Scalability Issue: many CPUs are accessing a shared data structure so locking can greatly reduce performance
- Cache Affinity Issue: frequently migrate cache data between CPUs.

MQMS: jobs are divided into multiple queues, one for each CPU.

Work Stealing: A source queue that is low on jobs take jobs from the other queue that has heavier jobs.

CFS Multiprocessor Features:

- Cache usage: when threads wakes up, if the difference of vruntimes is small, then do not preempted.
- load balancing:
 - Evens out work on all cores.
 - Every 4ms every core tries to steal work from other cores.
 - Tries to make balancing occurs within a NUMA node.

ULE Scheduler:

- ULE has three queues per core - one for interactive threads, one for batch threads, and one for idle threads. A thread's interactivity is calculated based on the last 5 seconds. Newly created threads inherit from their parent.
- Inside each queue, threads are sorted by priority (a combination of niceness and interactivity penalty for interactive queue, and niceness and runtime for batch queue). Inside of the queues, there is one FIFO per priority.
- When picking next thread, first search interactive queue then batch queue.

Distributed File Systems

Communication Protocols:

- UDP: Unreliable, connectionless, lightweight.
- TCP: Reliable, sequenced.

RPC: create wrappers so calling a function on another machine is like calling a local function.

Sun's Network File System (NFS): fast server crash recovery. The protocol was stateless (the server does not keep track of anything) and each request contains all the information needed.

Idempotent: the effect of performing the operation multiple times is equivalent to the effect of performing the operation once.

Cache Consistent: clients' cached data should be consistent with the server's data.

- Flash-on-close: in NFS, client checks server's file attributes before using cached copy.

AFS:

- Used whole-file caching on the local disk of client
- reads and writes are local
- Upon close, if changed, file is flushed to the server
- Subsequent accesses to file, will check if file on server has changed (TestAuth). If not, use local copy.
- Added a callback to reduce the number of TestAuth. Server will notify client if the file has been modified.
- Added file identifier so server only need to walk a path once.

Security

Security Goals:

- Confidentiality - If some piece of information is supposed to be hidden from others, don't allow them to find it out.
- Integrity - If some piece of information or component of a system is supposed to be in a particular state, don't allow an adversary to change it.
- Availability - If some piece of information or service is supposed to be available for your own or others' use, make sure an attacker cannot prevent its use.

Access control:

- Access Control Lists: each object has its own ACL of who is allowed to do what with the object.
- Capabilities: the OS maintains a set of capabilities for a process and as the process tries to access an object (e.g. open()), the OS checks that the appropriate capability "unlocks" access to the object.
- Discretionary Access Control: the owning user decides who has access to the object.
- Mandatory Access Control: some elements of access are decided by an authority who can override the desires of the owner.